

Algoritmi e Strutture Dati – Parte A – 25/08/25

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Supponiamo di dover scegliere tra i seguenti tre algoritmi:

- **Algoritmo A:** Risolve un problema di dimensione n dividendolo in 5 sottoproblemi di dimensione $n/2$, risolvendo ricorsivamente ciascun sottoproblema e combinando le soluzioni in tempo lineare.
- **Algoritmo B:** Risolve un problema di dimensione n risolvendo ricorsivamente tre sottoproblemi di dimensione $n - 10$ e combinando le soluzioni in tempo costante.
- **Algoritmo C:** Risolve un problema di dimensione n dividendolo in 16 sottoproblemi di dimensione $n/4$, risolvendo ricorsivamente ciascun sottoproblema e combinando le soluzioni in tempo $O(n^2)$.

Determinare i tempi di esecuzione di ciascun algoritmo e discutere quale algoritmo sceglieresti e perché.

A2 – Tassografia – Punti ≥ 10

Dato in input un grafo orientato $G = (V, E)$, scrivere un algoritmo

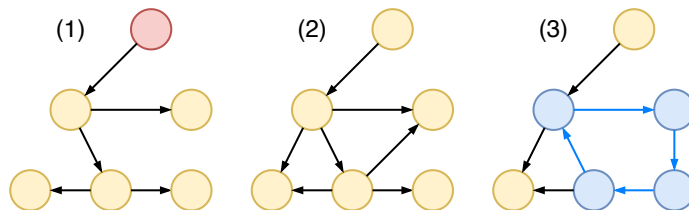
```
int categorizeGraph(GRAPH G)
```

che restituisca:

- **1** se G è un *arborescenza*, cioè un albero radicato orientato dalla radice ai figli in cui ogni nodo tranne la radice ha esattamente un arco entrante proveniente dal padre;
- **2** G è un *grafo diretto aciclico* (DAG) ma non un'arborescenza;
- **3** G non è un DAG.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale. Verranno dati punti parziali per l'identificazione di un sottoinsieme dei casi.

La figura seguente mostra un esempio per ognuno dei tre casi. Nel primo caso, la radice è segnata in rosa; nel terzo, il ciclo è evidenziato in azzurro.



A3 – Scendere nell'albero – Punti ≥ 12

Sia T un albero binario perfetto con $n = 2^d - 1$ nodi per qualche intero $d \geq 1$. Ogni nodo v di T è etichettato con un numero intero $x.val$. Si assuma che tutti i valori siano *distinti*.

Un nodo v di T è un *minimo locale* se il suo valore è strettamente minore dei valori di tutti i nodi ad esso adiacenti (ovvero il padre, se esiste, e i figli, se esistono):

Scrivere un algoritmo

```
TREE localMinimum(TREE T)
```

che prenda in input un albero T e restituisca uno qualunque dei minimi locali.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Nell'albero seguente, i nodi azzurri sono minimi locali. Uno qualunque di essi può essere restituito come output dell'algoritmo.

